K.R. Mangalam University

School of Engineering & Technology

Operating System Lab (ENCA252)

Lab Assignment 3

Comprehensive Implementation of Page Replacement Algorithms

Submitted by:

Name: Aditya Shibu

Roll Number: 2401201047

Course: BCA (AI & DS)

Submitted To:

Dr. Jyoti Yadav

OS Lab Assignment 3 - Page Replacement Algorithms Report

□ Cover Section

- **Name:** Aditya Shibu
- **Roll Number:** 2401201047
- **Course:** BCA (AI & DS)
- **University:** K.R. Mangalam University, School of Engineering & Technology
- **Subject:** Operating System Lab (ENCA252)
- **Submitted To:** Dr. Jyoti Yadav
- **Date:** April 19, 2026

□ Objective

Implement and analyze five major page replacement algorithms: **FIFO**, **LRU**, **Optimal**, **MRU**, and **Second Chance (Clock)**. The goal is to simulate memory behavior and evaluate performance based on **page faults**.

> **Note**
>
> Sample used throughout: Reference String — `7 3 0 3 2 1 2 0 7 0 1 2`, Frames — `3`

1. FIFO (First-In, First-Out)

How it works:

The oldest page in memory (the one that arrived first) is replaced when a new page needs to be loaded and all frames are full.

Logic:

A simple queue is maintained. When a page fault occurs and frames are full, the page at the front of the queue (oldest) is evicted and the new page is added to the rear.

Step-by-Step Trace

Step	Page	Frame 1	Frame 2	Frame 3	Page Fault
1	7	7	-	-	☑ Yes
2	3	7	3	-	☑ Yes
3	0	7	3	0	☑ Yes
4	3	7	3	0	✖ No
5	2	3	0	2	☑ Yes
6	1	0	2	1	☑ Yes
7	2	0	2	1	✖ No
8	0	0	2	1	✖ No
9	7	2	1	7	☑ Yes
10	0	1	7	0	☑ Yes
11	1	1	7	0	✖ No
12	2	7	0	2	☑ Yes

Total Page Faults: 8

💻 Code Output

```
~/Documents/Sem4/Os/Assignments/page-replacement-algorithms main
❯ python main.py
Enter page reference string (space separated): 7 3 0 3 2 1 2 0 7 0 1 2
Enter number of frames: 3

--- FIFO Page Replacement ---
Step  1: Page 7 → Frames: [7]             | Status: MISS
Step  2: Page 3 → Frames: [7, 3]          | Status: MISS
Step  3: Page 0 → Frames: [7, 3, 0]       | Status: MISS
Step  4: Page 3 → Frames: [7, 3, 0]       | Status: HIT
Step  5: Page 2 → Frames: [3, 0, 2]       | Status: MISS
Step  6: Page 1 → Frames: [0, 2, 1]       | Status: MISS
Step  7: Page 2 → Frames: [0, 2, 1]       | Status: HIT
Step  8: Page 0 → Frames: [0, 2, 1]       | Status: HIT
Step  9: Page 7 → Frames: [2, 1, 7]       | Status: MISS
Step 10: Page 0 → Frames: [1, 7, 0]       | Status: MISS
Step 11: Page 1 → Frames: [1, 7, 0]       | Status: HIT
Step 12: Page 2 → Frames: [7, 0, 2]       | Status: MISS
```

2. LRU (Least Recently Used)

How it works:

The page that has not been used for the longest period of time is replaced.

Logic:

Usage history is tracked for each page in memory. On a page fault, the page with the oldest last-use timestamp is evicted. Exploits **temporal locality** — recently used pages are likely to be used again soon.

> **Tip**
>
> LRU generally performs close to Optimal and is widely used in practice for its balance of efficiency and implementability.

Step-by-Step Trace

Step	Page	Frame 1	Frame 2	Frame 3	Page Fault
1	7	7	-	-	☑ Yes
2	3	7	3	-	☑ Yes
3	0	7	3	0	☑ Yes
4	3	7	0	3	✖ No
5	2	0	3	2	☑ Yes
6	1	3	2	1	☑ Yes
7	2	3	1	2	✖ No
8	0	1	2	0	☑ Yes
9	7	2	0	7	☑ Yes
10	0	2	7	0	✖ No
11	1	7	0	1	☑ Yes
12	2	0	1	2	☑ Yes

Total Page Faults: 9

💻 Code Output

```
--- LRU Page Replacement ---
Step  1: Page 7 → Frames: [7]             | Status: MISS
Step  2: Page 3 → Frames: [7, 3]          | Status: MISS
Step  3: Page 0 → Frames: [7, 3, 0]       | Status: MISS
Step  4: Page 3 → Frames: [7, 0, 3]       | Status: HIT
Step  5: Page 2 → Frames: [0, 3, 2]       | Status: MISS
Step  6: Page 1 → Frames: [3, 2, 1]       | Status: MISS
Step  7: Page 2 → Frames: [3, 1, 2]       | Status: HIT
Step  8: Page 0 → Frames: [1, 2, 0]       | Status: MISS
Step  9: Page 7 → Frames: [2, 0, 7]       | Status: MISS
Step 10: Page 0 → Frames: [2, 7, 0]       | Status: HIT
Step 11: Page 1 → Frames: [7, 0, 1]       | Status: MISS
Step 12: Page 2 → Frames: [0, 1, 2]       | Status: MISS
```

3. Optimal Page Replacement

How it works:

Replaces the page that will **not be used for the longest time in the future**.

Logic:

Looks ahead in the reference string to determine which page currently in memory will be needed farthest in the future (or not at all), and evicts that page. This is a theoretical algorithm — it cannot be implemented in real systems as it requires future knowledge.

> **Note**
>
> Optimal serves as a **benchmark** — no algorithm can produce fewer page faults than Optimal for the same input.

Step-by-Step Trace

Step	Page	Frame 1	Frame 2	Frame 3	Page Fault
1	7	7	-	-	☑ Yes
2	3	7	3	-	☑ Yes
3	0	7	3	0	☑ Yes
4	3	7	3	0	✖ No
5	2	7	2	0	☑ Yes
6	1	1	2	0	☑ Yes
7	2	1	2	0	✖ No
8	0	1	2	0	✖ No
9	7	1	7	0	☑ Yes
10	0	1	7	0	✖ No
11	1	1	7	0	✖ No
12	2	2	7	0	☑ Yes

Total Page Faults: 7

💻 Code Output

```
--- Optimal Page Replacement ---
Step  1: Page 7 → Frames: [7]             | Status: MISS
Step  2: Page 3 → Frames: [7, 3]          | Status: MISS
Step  3: Page 0 → Frames: [7, 3, 0]       | Status: MISS
Step  4: Page 3 → Frames: [7, 3, 0]       | Status: HIT
Step  5: Page 2 → Frames: [7, 2, 0]       | Status: MISS
Step  6: Page 1 → Frames: [1, 2, 0]       | Status: MISS
Step  7: Page 2 → Frames: [1, 2, 0]       | Status: HIT
Step  8: Page 0 → Frames: [1, 2, 0]       | Status: HIT
Step  9: Page 7 → Frames: [1, 7, 0]       | Status: MISS
Step 10: Page 0 → Frames: [1, 7, 0]       | Status: HIT
Step 11: Page 1 → Frames: [1, 7, 0]       | Status: HIT
Step 12: Page 2 → Frames: [2, 7, 0]       | Status: MISS
```

4. MRU (Most Recently Used)

How it works:

The **most recently used** page is replaced when a page fault occurs.

Logic:

Tracks which page was accessed last. On a fault, that most-recently-used page is evicted. This is the opposite of LRU and typically performs poorly for general workloads, but can be efficient for certain access patterns (e.g., sequential scans).

> **Tip**
>
> MRU can outperform LRU in specific scenarios such as cyclic sequential scans, where recently used pages are unlikely to be reused immediately.

Step-by-Step Trace

Step	Page	Frame 1	Frame 2	Frame 3	Page Fault
1	7	7	-	-	☑ Yes
2	3	7	3	-	☑ Yes
3	0	7	3	0	☑ Yes
4	3	7	3	0	✖ No
5	2	7	2	0	☑ Yes
6	1	7	1	0	☑ Yes
7	2	7	2	0	☑ Yes
8	0	7	2	0	✖ No
9	7	7	2	0	✖ No
10	0	0	2	0	✖ No
11	1	0	2	1	☑ Yes
12	2	0	2	1	✖ No

Total Page Faults: 7

💻 Code Output

```
--- MRU Page Replacement ---
Step  1: Page 7 → Frames: [7]             | Status: MISS
Step  2: Page 3 → Frames: [7, 3]          | Status: MISS
Step  3: Page 0 → Frames: [7, 3, 0]       | Status: MISS
Step  4: Page 3 → Frames: [7, 3, 0]       | Status: HIT
Step  5: Page 2 → Frames: [7, 2, 0]       | Status: MISS
Step  6: Page 1 → Frames: [7, 1, 0]       | Status: MISS
Step  7: Page 2 → Frames: [7, 2, 0]       | Status: MISS
Step  8: Page 0 → Frames: [7, 2, 0]       | Status: HIT
Step  9: Page 7 → Frames: [7, 2, 0]       | Status: HIT
Step 10: Page 0 → Frames: [7, 2, 0]       | Status: HIT
Step 11: Page 1 → Frames: [7, 2, 1]       | Status: MISS
Step 12: Page 2 → Frames: [7, 2, 1]       | Status: HIT
```

5. Second Chance (Clock Algorithm)

How it works:

A modified version of FIFO. Each page has a **reference bit**. When a page is accessed, its bit is set to 1. On replacement, if the oldest page's bit is 1, it gets a "second chance" (bit reset to 0, moved to back of queue); if 0, it is replaced.

Logic:

Maintains a circular queue (clock). The "clock hand" sweeps through pages. A page with reference bit = 1 is spared once; on the next sweep if still unreferenced, it is evicted. Approximates LRU with lower overhead.

> **Note**
>
> The Second Chance algorithm is used in real OS implementations (e.g., Linux) as an approximation of LRU without the full overhead of tracking timestamps.

Step-by-Step Trace *(Bits shown in parentheses: 1 = referenced)*

Step	Page	Frame 1	Frame 2	Frame 3	Page Fault
1	7	7(1)	-	-	☑ Yes
2	3	7(1)	3(1)	-	☑ Yes
3	0	7(1)	3(1)	0(1)	☑ Yes
4	3	7(1)	3(1)	0(1)	✖ No
5	2	2(1)	3(1)	0(1)	☑ Yes
6	1	2(1)	3(1)	1(1)	☑ Yes
7	2	2(1)	3(1)	1(1)	✖ No
8	0	2(1)	0(1)	1(1)	☑ Yes
9	7	2(1)	0(1)	7(1)	☑ Yes
10	0	2(1)	0(1)	7(1)	✖ No
11	1	1(1)	0(1)	7(1)	☑ Yes
12	2	1(1)	0(1)	2(1)	☑ Yes

Total Page Faults: 9

💻 Code Output

```
--- Second Chance (Clock) Page Replacement ---
Step  1: Page 7 → Frames: [7]             | Status: MISS
Step  2: Page 3 → Frames: [7, 3]          | Status: MISS
Step  3: Page 0 → Frames: [7, 3, 0]       | Status: MISS
Step  4: Page 3 → Frames: [7, 3, 0]       | Status: HIT
Step  5: Page 2 → Frames: [2, 3, 0]       | Status: MISS
Step  6: Page 1 → Frames: [2, 3, 1]       | Status: MISS
Step  7: Page 2 → Frames: [2, 3, 1]       | Status: HIT
Step  8: Page 0 → Frames: [2, 0, 1]       | Status: MISS
Step  9: Page 7 → Frames: [2, 0, 7]       | Status: MISS
Step 10: Page 0 → Frames: [2, 0, 7]       | Status: HIT
Step 11: Page 1 → Frames: [1, 0, 7]       | Status: MISS
Step 12: Page 2 → Frames: [1, 0, 2]       | Status: MISS
```

📊 Performance Comparison

Algorithm	Total Page Faults	Efficiency Rank
Optimal	7	1st (Theoretical minimum)
MRU	7	1st (Best real-world for this string)
FIFO	8	3rd
LRU	9	4th
Second Chance	9	4th

💻 Code Output

```
========================================
Algorithm            | Page Faults
----------------------------------------
FIFO                 | 8
LRU                  | 9
Optimal              | 7
MRU                  | 7
Second Chance        | 9
========================================
```

☑ Conclusion

- **Best Performer: Optimal** and **MRU** both achieved 7 page faults — the lowest. Optimal is the theoretical minimum benchmark; MRU matched it coincidentally due to the specific access pattern of this reference string.
- **Worst Performers: LRU** and **Second Chance** both had 9 faults for this input, demonstrating that performance is highly input-dependent and no single algorithm dominates all cases.
- **Observations: FIFO** sits in the middle at 8 faults. Interestingly, **Second Chance** — which is designed to improve on FIFO — performed worse here (9 vs 8), showing that the reference bit mechanism can backfire on short, non-repeating sequences. In production OS design, **LRU approximations** (like Second Chance/Clock) are still the most common choice overall due to their balance of practicality and average-case performance across diverse workloads.